Unicode text in ADA

John-Eric Söderman

Table of Contents

1 ADA as ADA ?....2
2 Wide....2
3 Basic....3
3.1 Issues....4
3.1.1 Preprocessor....4
3.1.2 Unicode....4
4 Intermediate....4
4.1 Y2018.Text and solving text issues in ADA....4
4.1.1 Building the base for text....5
4.1.2 String – Array (<>) of Character....5
4.1.3 Wide_String – Array (<>) of Wide Character....5
4.1.4 Wide_Wide_String – Array (<>) of Wide Wide Character....5
4.2 Wide_Wide_String – Array (<>) of Wide Wide Character but much more....6
4.3 Using “+” instead of “&”....6
4.3.1 Package Y2018.Text.Wide.Tool....7
4.3.2 Working without preprocessor....7
4.3.3 Package Y2018.Text.Core.Tool....8
4.4 Regular expression....8
4.4.1 Program pattern for search....9
4.4.2 Y2018.Text.Regex.PatternPack....11
4.4.3 Y2018.Text.Regex.MatchPack....12
4.5 Examples that do not compile....13
4.6 The preprocessor *Y2018Text*....14
4.7 Conclusions of this chapter....15
4.7.1 Comparing Ada with Perl....15
5 Make....15
5.1 What work-directory contains....16

1 ADA as ADA ?

The source code of the sample.adb

```
17|with Ada.Text_IO; use  Ada.Text_IO;
18|with Ada.Strings.Unbounded; use Ada.Strings.Unbounded;
19|with Y2018.Text.Wide; use Y2018.Text.Wide;
20|with Y2018.Text.Wide.STR; use Y2018.Text.Wide.STR;
21|with Y2018.Text.Wide.Tool;
22|with Y2018.Text.Regex.PatternPack;
23|-- with DSECT;
24|procedure Sample is
25|  s:Unbounded_String;
26|  HULK:constant String:="Hulk";
27|  r:Tool.M_AAAC;
28|  p:Y2018.Text.Regex.PatternPack.Pattern_AC:=
29|    Y2018.Text.Regex.PatternPack.compile(
30|      "^(.)([\w]+)"W,
31|      Y2018.Text.Regex.PatternPack.G_NONE
32|    );
33|begin
34|  Put_Line ("--- Begin of Sample ---");
35|  s:=NUS + HULK + "ior"W;
36|  Put_Line ("" + "WIDE_WIDE "W + s);
37|  if Tool.matchTo("What",p,r) then
38|    for i in r'First .. r'Last loop
39|      Put_Line (
40|        ">" + Integer'Image(i) +
41|        "=" + r(i).all + "<"
42|      );
43|    end loop;
44|  end if;
45|   Put_Line ("*** End of Sample ***");
46|end Sample;
```

Something strange about the “ADA code” ?

To create this, see Chapter Make, page 15 .

2 Wide

Lets look at line 35 of the source code at page 2 . The “ior”W is a notation for a literal of type Wide_Wide_String containing three Wide_Wide_Characters, ‘i’W, ‘o’W and ‘r’W. Today there are no such in ADA but why not ? Then we have the use of “+” operator in a strange way. It is not possible to overload “&”-operator differently because of the definitions in the standard ADA packages, we use the “+” operator as a stand-in for “&”-operator.
Same line but next question. HULK is defined as constant String(1..4) but “ior”W is constant Wide_Wide_String(1..3). The “+”-operator joins these two ?? NUS is defined in Y2018.Text.Wide

as Unbounded.Unbounded_String with a value Null_Unbounded_String. The result to "s" is assigned

1. NUS
2. HULK but converted to Unbounded_String
3. "ior"W converted to UTF-8 string (= standard ADA String) and converted to Unbounded_String

The "+" operator takes two arguments and convert the result to an array type of the first argument is the first argument is not an array type. Unbounded_String and Unbounded_Wide_Wide_String are considered to be array types here although not in standard ADA,
All these conversions is controlled by definitions specified at line 20. These switching between unbounded and standard is the reason why "&"-operator can not be used.

These four constants are used as (type-) casts in string "+"-operator expressions

- "" – empty String literal
- ""W – empty Wide_Wide_String literal
- NUS – Null Unsigned String constant
- NUW – Null Unsigned Wide_Wide_String constant

3 Basic

Line 37 on page 2 contains a function called Tool.matchTo with three arguments

1. A standard ADA string, "What"
2. "p" a pattern
3. "r" of type Tool.MAAAC

We are matching a string ("What") to a regular expression "^(.)([\w]+)". If matching succeed we study it at lines 38..43 else we jump to the line 44.

The pattern is defined at lines 29..32 by invoking Regex.Patternpack("^(.)([\w]+)", Regex.Patternpack.G_NONE). First argument is the regular expression and second is saying that no options is used.

Tool.MAAAC is output from Tool.matchTo function and contains

- index 0 – found expression, in this case "What"
- index 1 – first parentheses expression "W"
- index 2 – second parentheses expression "hat"

Tool.MAAAC has these values as Unbounded_Wide_Wide_String access types in an array. At line 41 we get the value in the expression "r(i).all" (all is an ADA solution to get access of all the content of an access type). By using the "+" operator (see page 2) we get the value converted to standard ADA string.
Values in ToolMAAAC follows the regular expression rule that index zero is the whole expression and from index one forwards we get the content of the parentheses expressions with capturing.

3.1 Issues

3.1.1 Preprocessor

Source code with mixed Wide_Wide_String and String literals must be preprocessed. Source code using Package Y2018.Text.Wide must also be preprocessed. Preprocessor (see chapter “The preprocessor Y2018Text” on page 14) convertion of string-literals has some issues

1. Functions with operator (“+”, “and” ...) name has special handling
    1. function “+” (… ===> operator name after key-word ‘function’ is not converted
    2. end “+”; ===> operator name after keys-word ‘end’ is not converted
    3. there are cases when the preprocessor should leave a string literal as is then a special syntax should be used (example: “+”S). One example is in generics with operator-functions
2. If source code is preprocessed then the prepocessor will use the DSECT comment for inserting the with-clause. If no DSECT comment is found an extra line is inserted in the code.

3.1.2 Unicode

Unicode 12 from Unicode.org is supported.

4 Intermediate

Text handling in ADA programming language is today not very well supported and a heap of strange issues makes it harder. By introducing a preprocessor for the ADA source code and by restricting the use of the use-clause a more user friendly ADA can be presented.
These solutions is supposing only the GNAT implementation of ADA. On platforms with non-GNAT ADA implementation other measures may be used.

4.1 Y2018.Text and solving text issues in ADA

Most text issues can be solved through use of Regular Expression. There exists some tools for this in the Standard package but not a full implementation as in Perl. Ada has no virtual machine but is a language which is compiled to the native machine code. A regular expression implementation cannot be built for ADA as it is, we have to choose a specific ADA implementation, in this case we have chosen as base GNAT. With GNAT we get UTF representations of characters with specific binary representation.

The interface to the Regular Expression implementation differs from that of Perl although the syntax of the regular expression is almost the same and for anyone familiar with Perl regular expression will be familiar with this. This implementation is not based on ANSI but based on Unicode. A “[\w]” stands for any Unicode word character not ANSI word character. Translation from lower case to upper case cannot be done for all lower case characters because in Unicode all lower case characters do not have an upper case equivalent, and many characters are not lower or upper case at all. The policy in case translation is that it is done if there exists equivalent character and the translation is one to one. But even this can generate errors, because in Turkish the translation will convert <u>i with dot</u> to <u>I without a dot</u>.
Support is for the full Unicode, not parts. Unicode representation needs 21 bits for a character value and this is supported.

In ADA 2012 for GNAT has from a regular expression viewpoint some issues

- Wide_Wide_String and Wide_Wide_Character literals
- Standard library contains &-operator overloading which is not as specified in ADA standard
- &-operator overloading is specified in different packages without any possibilities to change. It would be better to delete all these &-operator overloading functions from these standard packages
- Function and procedure calls cannot differentiate between constant expressions of Wide_Wide_String, Wide_String and String and between Wide_Wide_Character, Wide_Character and Character

In this presentation we use an ADA based preprocessor to change the source code to a form acceptable to the Gnat compiler. And use +-operator as a stand-in for &-operator.

4.1.1 Building the base for text

ADA language has three character types

1. Character
2. Wide Character
3. Wide Wide Character

in the GNAT implementation the solution is

1. Character – UTF-8 from Unicode.org
2. Wide Character – UTF-16BE from Unicode.org
3. Wide Wide Character – UTF-32BE from Unicode.org

A Unicode character also called code point is a entity specified for 21 bits. The result for ADA is that only Wide Wide Character has a one-to-one relation to a code point.

4.1.2 String – Array (<>) of Character

In UTF-8 code points are fitted in 8 bits. A code point stored as UTF-8 is 1 to 4 bytes. The result is a string with 4 characters stored as UTF-8 is 4..16 bytes long, and to access the fourth character it starts in byte …
In ADA a constant string can be specified with quotation notation, “A String”.

4.1.3 Wide_String – Array (<>) of Wide Character

UTF-16BE is more loosely packed that UTF-8 but do still have the problems with code points which cannot be fitted inside 16 bits. In Y2018.Text we made a decision not to even try to support UTF-16BE.

4.1.4 Wide_Wide_String – Array (<>) of Wide Wide Character

Because of the 1-1 relation between code point and character text operations on strings in UTF-32BE is most straight forward of all other UTF storage formats.
But ADA lacks literal syntax for Wide_Wide_Strings. In Y2018.Text preprocessor implementation we added Wide_Wide_String literal, “A String”W . My_variable:Wide_Wide_String:="A String"W; generated as:

```
My_variable:Wide_Wide_String:=(Wide_Wide_Character'Val(16#41#),
Wide_Wide_Character'Val(16#20#),Wide_Wide_Character'Val(16#53#),
Wide_Wide_Character'Val(16#74#),Wide_Wide_Character'Val(16#72#),
Wide_Wide_Character'Val(16#69#),Wide_Wide_Character'Val(16#6E#), Wide_Wide_Character'Val(16#67#));
```

but now we can access the fourth (wide wide) character with My_variable(4). (more about this issue at page 6)

4.2 Wide_Wide_String – Array (<>) of Wide Wide Character but much more

In Y2018.Text preprocessor implementation we added Wide_Wide_String constant, “A String”W .
My_variable:Wide_Wide_String:="A String"W; generated as:

```
My_variable:Wide_Wide_String:=(Wide_Wide_Character'Val(16#41#),
Wide_Wide_Character'Val(16#20#),Wide_Wide_Character'Val(16#53#),
Wide_Wide_Character'Val(16#74#),Wide_Wide_Character'Val(16#72#),
Wide_Wide_Character'Val(16#69#),Wide_Wide_Character'Val(16#6E#), Wide_Wide_Character'Val(16#67#));
```

But this is not all. Characters in a Wide_Wide_String literal is extended by escaping a character with a back slash

\\	(16#5C#)	back slash as back slash
\b	BS : 16#08#	backspace
\f	FF : 16#0C#	formfeed
\n	LF : 16#0A#	linefeed
\r	CR : 16#0D#	carriage return
\t	HT : 16#09#	horisontal tabulation
\x88	Character as hexadecimal	two hexadecimal digits
\u8888	Character as hexadecimal	four hexadecimal digits
\v888888	Character as hexadecimal	six hexadecimal digits
\y88888888	Character as hexadecimal	eight hexadecimal digits

To escape other characters with back slash is not allowed by the preprocessor.

4.3 Using “+” instead of “&”

In ADA “&” operator should be used to concatenate arrays, but errors in the standard ADA libraries make this impossible. There exists an altenative, use the “+” operator instead.

left\right argument	**String**	**Wide_Wide_String**	**Unbounded_Wide_Wide_String**	**Unbounded_String**	**Character**	**Wide_Wide_Character**
String	1 String	2 **[UTF-8]** String	3 **[UTF-8]** String	4 String	5 String	6 **[UTF-8]** String
Wide_Wide_String	7 **[UTF-32]** Wide_Wide_String	8 Wide_Wide_String	9 Wide_Wide_String	10 **[UTF-32]** Wide_Wide_String	11 **[UTF-32]** Wide_Wide_String	12 Wide_Wide_String
Unbounded_Wide_Wide_String	13 **[UTF-32]** Unbounded_Wide_Wide_String	14 Unbounded_Wide_Wide_String	15 Unbounded_Wide_Wide_String	16 **[UTF-32]** Unbounded_Wide_Wide_String	17 **[UTF-32]** Unbounded_Wide_Wide_String	18 Unbounded_Wide_Wide_String
Unbounded_String	19 Unbounded_String	20 **[UTF-8]** Unbounded_String	21 **[UTF-8]** Unbounded_String	22 Unbounded_String	23 Unbounded_String	24 **[UTF-8]** Unbounded_String
Character	25 String	26 **[UTF-8]** String	27 **[UTF-8]** String	28 String	29 String	30 **[UTF-8]** String
Wide_Wide_Character	31 **[UTF-32]** Wide_Wide_String	32 Wide_Wide_String	33 Wide_Wide_String	34 **[UTF-32]** Wide_Wide_String	35 **[UTF-32]** Wide_Wide_String	36 Wide_Wide_String

36 “+” overlay functions. With “&” functions (1) and (8) would be supported by ADA but with “+” such restrictions do not apply. In some cases a conversion between UTF-8 and UTF-32BE must be

applied. In the table first argument is as row-heading and second argument is as column-heading and the result is in the table element (with a number and a note about the need for conversion). ADA specification about overloading functions with two arguments is strictly followed, first argument decides the type for result array. This makes it possible to write

```
s:=Null_Unbounded_String + HULK + "ior"W;
```

Note the used programming pattern based on a null string

- “” - convert result to String
- “”W – convert result to Wide_Wide_String
- Null_Unbounded_String – convert result to Unbounded String
- Null_Unbounded_Wide_Wide_String – convert result to Unbounded Wide Wide String

No overlay support is built for Bounded Strings or Bounded Wide Wide Strings.

By defining all “+” overlay functions in one package and only these functions we are giving a programmer a choice. Overlaying is a powerful feature but there is cases when it should not be used.

4.3.1 Package Y2018.Text.Wide.Tool

Text processing needs more and package Y2018.Text.Wide.Tool contains some of the needed tooling (code see Appendix 3 at page Error: Reference source not found)

- To_Hex* – a group of functions for converting to hexadecimal digits
  - To_Hex8 - Convert value to eight hexadecimal characters with '_' as prefix padding, returns String
  - To_Hex8W - Convert value to eight hexadecimal characters with '_' as prefix padding, returns Wide_Wide_String
  - To_HexX - Convert value to hexadecimal characters, where result length is 2 characters
  - To_HexU - Convert value to hexadecimal characters, where result length is 4 characters
  - To_HexV - Convert value to hexadecimal characters, where result length is 6 characters
  - To_HexY - Convert value to hexadecimal characters, where result length is 8 characters
- Substr – retrieve sub-string of a string. “Substr” give start position and amount of characters
- Reindex – re index a string by setting 'First(1) to one
- CmpS – compare two strings by extending the shorter with space characters before comparing
- Right, Left and Center – shift or extend string to right, left or center the string
- Strip, RightStrip and LeftStrip – Strip a padding character from both, right or left
- To8 and To32 – convert between UTF-32 and UTF-8
- Get_GC – get Unicode.org general class code
- Get_UC, Get_LC and Get_TC – convert to uppercase, lowercase and titlecase

4.3.2 Working without preprocessor

To work without the preprocessor the &-operator should be used and no Wide_Wide_String literals (Code see Appendix 1 at page Error: Reference source not found). Note the limited support but these overlays will work without any preprocessing.

left\right argument	**Wide_Wide_String**	**Unbounded_Wide_Wide_String**	**Unbounded_String**	**Wide_Wide_Character**
Wide_Wide_String	Wide_Wide_String	Wide_Wide_String	**[UTF-32]** Wide_Wide_String	Wide_Wide_String
Unbounded_Wide_Wide_String	Unbounded_Wide_Wide_String	Unbounded_Wide_Wide_String	**[UTF-32]** Unbounded_Wide_Wide_String	Unbounded_Wide_Wide_String
Unbounded_String	**[UTF-8]** Unbounded_String	**[UTF-8]** Unbounded_String	Unbounded_String	**[UTF-8]** Unbounded_String
Wide_Wide_Character	Wide_Wide_String	Wide_Wide_String	**[UTF-32]** Wide_Wide_String	Wide_Wide_String

Compare this table to the table in chapter Using “+” instead of “&” at page 6 .

4.3.3 Package Y2018.Text.Core.Tool

A limited Y2018.Text.Wide.Tool, “Y2018.Text.Core.Tool”. This package will work without preprosessing. It seems to be like the corresponding Wide.Tool but it is not, because String and Character types are not supported here.

4.4 Regular expression

Package Y2018.Text.Regex contains an implementation of Regular expression in ADA. This implementation is based on UTF-32 and Wide_Wide_Character of ADA. The package Y2018.Text.Regex is the interface to the implementation not the implementation.

type	value	symbol
`MvertLine : constant MCHAR`	`Wide_Wide_Character'Val(Mlow_Integer +0)`	`|`
`MleftPal : constant MCHAR`	`Wide_Wide_Character'Val(Mlow_Integer +1)`	`(`
`MrightPal : constant MCHAR`	`Wide_Wide_Character'Val(Mlow_Integer +2)`	`)`
`MleftSquare : constant MCHAR`	`Wide_Wide_Character'Val(Mlow_Integer +3)`	`[`
`MrightSquare : constant MCHAR`	`Wide_Wide_Character'Val(Mlow_Integer +4)`	`]`
`MleftCurly : constant MCHAR`	`Wide_Wide_Character'Val(Mlow_Integer +5)`	`{`
`MrightCurly : constant MCHAR`	`Wide_Wide_Character'Val(Mlow_Integer +6)`	`}`
`Mcircumflex : constant MCHAR`	`Wide_Wide_Character'Val(Mlow_Integer +7)`	`^`
`Mdollar : constant MCHAR`	`Wide_Wide_Character'Val(Mlow_Integer +8)`	`$`
`Masterix : constant MCHAR`	`Wide_Wide_Character'Val(Mlow_Integer +9)`	`*`
`Mplus : constant MCHAR`	`Wide_Wide_Character'Val(Mlow_Integer +10)`	`+`
`Mquestion : constant MCHAR`	`Wide_Wide_Character'Val(Mlow_Integer +11)`	`?`
`Mperiod : constant MCHAR`	`Wide_Wide_Character'Val(Mlow_Integer +12)`	`.`
`Mlc_d : constant MCHAR`	`Wide_Wide_Character'Val(Mlow_Integer +13)`	`d`
`Muc_d : constant MCHAR`	`Wide_Wide_Character'Val(Mlow_Integer +14)`	`D`
`Mlc_s : constant MCHAR`	`Wide_Wide_Character'Val(Mlow_Integer +15)`	`s`
`Muc_s : constant MCHAR`	`Wide_Wide_Character'Val(Mlow_Integer +16)`	`S`
`Mlc_w : constant MCHAR`	`Wide_Wide_Character'Val(Mlow_Integer +17)`	`w`
`Muc_w : constant MCHAR`	`Wide_Wide_Character'Val(Mlow_Integer +18)`	`W`
`Mlc_p : constant MCHAR`	`Wide_Wide_Character'Val(Mlow_Integer +19)`	`p`
`Muc_p : constant MCHAR`	`Wide_Wide_Character'Val(Mlow_Integer +20)`	`P`

To insert any of the above listed non-alphabetic characters in a pattern you have to back slash these (“|”,”(“,”)”,”[“,”]”,”{“,”}”,”^”,”$”,”*”,”+”,”?” and ”.”). Back slash (“\”) must also be escaped with a back slash.
Symbol “p” and “P” is supported only for Unicode general classes.
Note that division (“/”) is not in the list above.

4.4.1 Program pattern for search

The task is to search through the “source data” with a pattern not ending with “\$” symbol, and looking for text and ignoring case (see also picture Search on page 11).

```
[01] with Ada.Text_IO; use  Ada.Text_IO;
[02] with Ada.Strings.Wide_Wide_Unbounded; use Ada.Strings.Wide_Wide_Unbounded;
[03] with Y2018.Text.C; use Y2018.Text.C;
[04] with Y2018.Text.C.STR; use Y2018.Text.C.STR;
[05] with Y2018.Text.C.UTF; use Y2018.Text.C.UTF;
[06] with Y2018.Text.C.Tool; use Y2018.Text.C.Tool;
[07] with Y2018.Text.Regex; use Y2018.Text.Regex;
[08] with Y2018.Text.Regex.PatternPack; use Y2018.Text.Regex.PatternPack;
[09] with Y2018.Text.Regex.MatchPack; use Y2018.Text.Regex.MatchPack;
[10] -- with DSECT;
[11] procedure Alfa is
[12]   pv:Wide_Wide_String:="^[\s]*[\w]+"W; -- pattern value
[13]   p:PatternPack.Pattern_AC:=
[14]      PatternPack.compile(pv,PatternPack.G_UPPERCASE);
[15]   myCursor: PatternPack.G_Cursor;
[16]   m:MatchPack.Match_AC;
[17]   s:Wide_Wide_String:="Kajsa Kalle Knatte Fnatte"W; -- source data
[18]   begin
[19]   myCursor:=PatternPack.matches(p,s);
[20]   while PatternPack.has_Element(myCursor) loop -- outer
[21]      m:=PatternPack.element(myCursor);
[22]      declare
[23]         aa:Unbounded_Wide_Wide_String_AA:=MatchPack.getMatch(m);
[24]      begin
[25]         for j in aa'First .. aa'Last loop -- inner
[26]            Ada.Text_IO.Put_Line
[27]            (
[28]               (
[29]                  UTF.To8
[30]                  (
[31]                     To_Wide_Wide_String
[32]                     (
[33]                        aa(j).all
[34]                     )
[35]                  )
[36]               )
[37]            );
[38]         end loop; -- inner
[39]      end;
[40]      myCursor:=PatternPack.next(myCursor);
[41]   end loop; -- outer
[42] end Alfa;
```

But before you start to use this solution you have to ask yourself do I need such a complicated solution ? Why not use the solution in the Sample, see page 2 .

The solution is to use two loops, outer loop for matching and inner loop for retrieve found items in the match result.

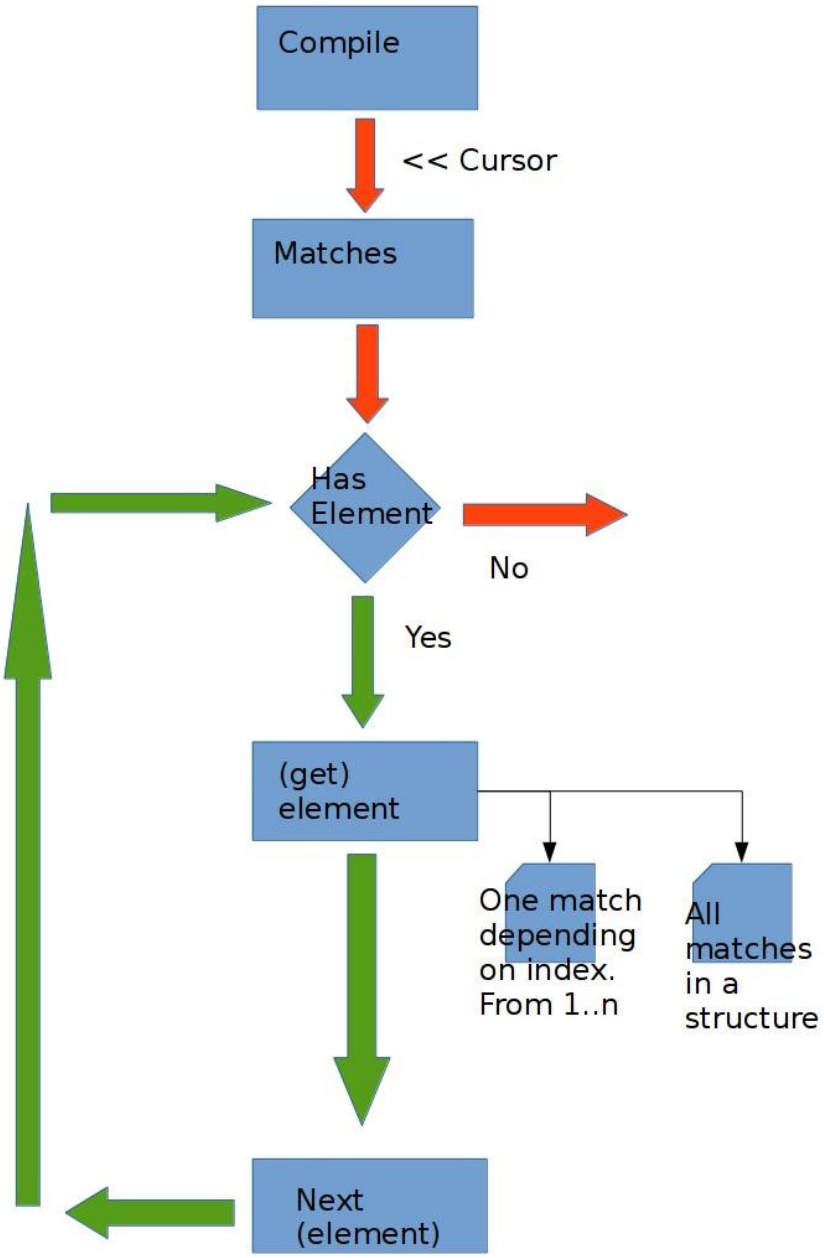
- Line [13] and [14] creates the pattern
- Line [15] defines the PatternPack specific cursor, the cursor of G_Cursor type
- Line [16] declares the matcher
- Line [17] the source data as a Wide_Wide_String
- Line [19] retrieves the cursor for the pattern and first match
- Line [20] checks if a match (element) exists
- Line [23] contains the **aa** item of Unbounded_Wide_Wide_String_AA type ,this is a array access type which makes it possible to get all match results in one call. To access a specific item see line [24] and line [17]
- Lines [29] to [35] converts Unbounded Wide Wide String to String
- Line [40] uses the pattern again to get the next match

Procedure Alfa publishes on output the results of the search.

Some of the reasons to use the 'intermediate' solution:

1. We are scanning through a text with a pattern and we want to continue scanning after the pattern is found (line [40] above)
2. We want to know if a result of zero length is an indicator of a zero length result or a no match situation [`function hasMatch (matchValue:Match_AC; mid:Integer) return Boolean;`]
3. We want access to the data after a found pattern [`function source (Position : G_Cursor) return Wide_Wide_String`]

```
package Y2018.Text.Regex.MatchPack is
 ...
 type Unbounded_Wide_Wide_String_AC is access all
Ada.Strings.Wide_Wide_Unbounded.Unbounded_Wide_Wide_String;
 type Unbounded_Wide_Wide_String_AA is array (Integer range <>) of
Unbounded_Wide_Wide_String_AC;
 ...
```



Picture **Search**

4.4.2 Y2018.Text.Regex.PatternPack

As can be seen we uses two packages for the implementation of regular expression.
Package Patternpack creates the pattern by compiling the pattern source. As the result we get pattern (an access type to the pattern). Pattern is input to the function matches. Matches returns the G_Cursor.

G_Cursor points to the start for next search and we can query what is next by using the source function. G_Cursor moves allays forward in a matching operation (left to right).

```
with Ada.Strings.Wide_Wide_Unbounded; use Ada.Strings.Wide_Wide_Unbounded;
package Y2018.Text.Regex.PatternPack is
```

```
type Pattern is tagged private;
type Pattern_AC is access Pattern;
type G_Cursor is private;
type G_Option is (G_NONE,G_UPPERCASE);
G_No_Element:constant G_Cursor;
function has_Element(Position:G_Cursor)return Boolean;
function matches(ptc:Pattern_AC;s:Wide_Wide_String)return G_Cursor;
function next(Position:G_Cursor)return G_Cursor;
function element(Position:G_Cursor)return Y2018.Text.Regex.MatchPack.Match_AC;
function source(Position:G_Cursor)return Wide_Wide_String;
function compile(s:Wide_Wide_String)return Pattern_AC;
function compile(s:Wide_Wide_String;option:G_Option)return Pattern_AC;
function compileC(s:Wide_Wide_String;option:G_Option)return Pattern_AC;
end Y2018.Text.Regex.PatternPack;
```

Functions:

- has_Element() - do the G_Cursor contain a match result
- matches(,) - do the match and return a G_Cursor
- next() - do next match and return a new G_Cursor
- element() - get the match result
- source() - get source data for next match operation
- compile() - compile pattern source and return the access type to the pattern
- compile(,) - compile pattern source and return the access type to the pattern. Second parameter sets option for the compilation.
- CompileC(,) - advanced pattern compilation

4.4.3 Y2018.Text.Regex.MatchPack

Retrieve the match result.
Two different solutions

1. Retrieve results as result of a specified expression using a index. Index value zero retrieves all characters from a match (`function getMatch(matchValue:Match_AC;mid:Integer) return Wide_Wide_String;`). The mid is the index and in range 0 .. n, where zero stands for all and 1..n is individual parenthesis expressions. If no corresponding expression exists or the expression do not have a value a Wide_Wide_String of zero length is returned. Function `hasMatch(matchValue:Match_AC;mid:Integer) return Boolean` can be used for determining if the index do have an expression and function `count(matchValue:Match_AC) return Integer` returns the number of expressions in the match
2. Retrieve all results as a Unbounded Wide Wide String AA variable (`function getMatch(matchValue:Match_AC) return Unbounded_Wide_Wide_String_AA;`) The "Unbounded Wide Wide String AA" variable is an array of access types and each item contains a Unbounded Wide Wide String, possibly of zero length. Number of items is the number of specified expressions and a item at index value zero (all source data)

Solution 2 is used in the example.

```
with Ada.Strings.Wide_Wide_Unbounded;
package Y2018.Text.Regex.MatchPack is
type Match is tagged private;
type Match_AC is access Match;
type Unbounded_Wide_Wide_String_AC is access all
Ada.Strings.Wide_Wide_Unbounded.Unbounded_Wide_Wide_String;
type Unbounded_Wide_Wide_String_AA is array(Integer range <>)of
Unbounded_Wide_Wide_String_AC;
function new_Match (jAC:Y2018.Text.Jets.MatchPack.Match_AC;
us:Ada.Strings.Wide_Wide_Unbounded.Unbounded_Wide_Wide_String)return Match_AC;
function getMatch(matchValue:Match_AC;mid:Integer)return Wide_Wide_String;
```

```
function getMatch(matchValue:Match_AC)return Unbounded_Wide_Wide_String_AA;
function hasMatch(matchValue:Match_AC;mid:Integer)return Boolean;
function count(matchValue:Match_AC)return Integer;
end Y2018.Text.Regex.MatchPack;
```

Functions:

- new_Match(,) - (internal use, do not call)
- getMatch(,) - get result as single expression, solution 1
- getMatch() - get all result, solution 2
- hasMatch(,) - check if an index has a matching expression, solution 1
- count() - get number of all expression items, usually in solution 1

.

4.5 Examples that do not compile

Why do not this compile ?

```
with Ada.Text_IO;
procedure Problem is
   function get(right:String) return String is
   begin
      return "get S";
   end get;
   function get(right:Wide_Wide_String) return String is
   begin
      return "get WWS";
   end get;
begin
   Ada.Text_IO.Put_Line ( get("01"));
   Ada.Text_IO.Put_Line
( get((Wide_Wide_Character'Val(16#30#),Wide_Wide_Character'Val(16#32#))));
end Problem;
```

and why not this ??

```
with Ada.Text_IO;
procedure ProblemB is
   function get(right:String) return String is
   begin
      return "get S";
   end get;
   function get(right:Wide_Wide_String) return String is
   begin
      return "get WWS";
   end get;
begin
   Ada.Text_IO.Put_Line ( get((Character'Val(16#30#),Character'Val(16#31#))));
   Ada.Text_IO.Put_Line
( get((Wide_Wide_Character'Val(16#30#),Wide_Wide_Character'Val(16#32#))));
end ProblemB;
```

.. but this compiles

```
with Ada.Text_IO;
procedure ProblemC is
   function get(right:String) return String is
   begin
      return "get S";
   end get;
   function get(right:Wide_Wide_String) return String is
   begin
      return "get WWS";
   end get;
   first: String:=(Character'Val(16#30#),Character'Val(16#31#));
   second:
Wide_Wide_String(1..2):=(Wide_Wide_Character'Val(16#30#),Wide_Wide_Character'Val(1
6#32#));
```

```
begin
   Ada.Text_IO.Put_Line ( get(first));
   Ada.Text_IO.Put_Line ( get(second));
end ProblemC;
```

The problem seems to be something with overloading and String and Wide Wide String literals (same problem with Character and Wide Wide Character).
A working solution is to get rid of all String and Wide Wide String literals from all function or procedure calls.
To fix the problem we use the preprocessor of Y2018.Text. By dumping all literals to a different package and replacing constants from this package in function and procedure calls the compiler will behave as in the third workable example. The preprocessor must insert an extra line in the source code, a with clause, referring to the new extra package.
The name of the extra package depends on the original source code

- procedure - {name}_psect.ads
- package head - {name}_hsect.ads
- package body – {name}_bsect.ads

and the inserted line will be `with {name}_psect;` or `with {name}_hsect;` or `with {name}_bsect;`.
It important to specify these constants outside the original source to keep line numbering in error messages unchanged (almost). Position of the error at a line will be corrupted.

4.6 The preprocessor *Y2018Text*

The preprocessor Y2018Text is a procedure written in ADA. As input is the source code and as output one or two files, the processed source code and the extra file if any String or Wide Wide String literals is found.

To get the job done all lines must be checked for String, Wide_Wide String[1], Character and Wide Wide Character[2] literals and these must be removed to the different file and a constant variable must be inserted as a replacement.

A specific lookout must be done for main package or procedure name to make it possible to name the extra file and to find the place for the extra line to insert in the source code.

The preprocessor uses Y2018.Text regular expression to search lines from input data. Following items are searched for

- Operator overloading functions[1]
- Character literals ('C')
- String literals (“String”)
- Wide Wide Character literals ('C'W)
- Wide Wide String literals (“String”W)
- Comments
- Procedure[1] – to get the name of the main procedure
- Package[1] – to get the name of package header
- Package Body[1] – to get the name of the package body
- End[1] (of operator overloading functions)
- -- DSECT comment, reservation for extra insert line

1 Wide Wide String Y2018.Text style, “text”W
2 Wide Wide Character Y2018.Text style 'C'W

The preprocessor procedure works in two steps. First step make the search and writes two temporary files. Second step copies one of the files to the name of the source code and if literals was found then copies the second file to a *Dsect_ads* package (*Dsect_ads* is {name}_psect.ads if it is a procedure, {name}_hsect.ads if it is a package header or {name}_bsect.ads if it is a package body).

4.7 Conclusions of this chapter

Best known regular expression implementation is in Perl language, and mostly this implementation is considered to be the standard. Regular expression implementation here do not contain all features of the Perl implementation and the reason is the different work environment.
As opposed to Perl the base is UTF-32 and Unicode.org specification. The result is that handling of characters with a code-point above 16 bits is supported and character classes are based on Unicode specification not ANSI. The implementation can evolve as Unicode evolves with new specified characters.
The solution to get around ADA/Gnat 2012 issues can be obsolete if the next ADA standard contains measures to correct these.

4.7.1 Comparing Ada with Perl

ADA (our solution)

```
37|  if Tool.matchTo("What",p,r) then
38|    for i in r'First .. r'Last loop
39|      Put_Line (
40|        ">" + Integer'Image(i) +
41|        "=" + r(i).all + "<"
42|      );
43|    end loop;
44|  end if;
```

PERL

```
01|$w="What";
02|if($w =~/^(.)([\w]+)/)
03|{
04| for($i=0;$i<@-;$i++)
05| {
06|  print STDOUT (
07|  '>'.$i.
08|  '='.
09|substr($w,$-[$i],$+[$i] - $-[$i]).
10|  "<\n");
11| }
12|}
```

The sample from a Perl script is rather strange but the program pattern of both solutions are as equal as possible.
In Perl usually is used special variables $1,$2,... but for the comparation we are using special arrays “@-” and “@+” to mimic “r(i).all”.
The purpose of this is to show how similar an Ada solution can be to Perl code.

5 Make

To Do:
After exploding the archive file (*.tar or maybe *.zip) in an empty directory, we name it here as work-directory. Do following:

1. check that gnat exists on the computer. Run command “gnat --version” in a terminal window.
2. run make-utility in a terminal window in the work-directory without any parameters (write command “make”) – this will create the binary files
3. to check that result is successful run sample program with command “make run”

If successful you have a temporay installation of the product, but more work is neede if you want to use it for programming

- liby2018text.so.1.1.2 has to be installed from /lib
- ln -s liby2018text.so has to be created
- directory /ads must be copied

- directory /ali must be copied. All files must be set to read-only
- y2018text must be defined in PATH, maybe copied to another directory

Examples of how to do see the /makefile, as examples see following steps

- sample.adb:
  - export LD_LIBRARY_PATH="lib";bin/y2018text pre/sample.adb tmp/
  - mv tmp/sample* .
- sample: gnatmake -aIads -aOali sample.adb -largs -ly2018text -Llib
- run : export LD_LIBRARY_PATH="lib";./sample

Step sample.adb: preprocess the code with the preprocessor and move Ada results
Step sample.adb: compile with Gnat compiler
Step run: if liby2018text.so not in place you can use this solution but if liby2018text.so in place give only the command ./sample

5.1 What work-directory contains

(after make is done)
ads – Ada ads-files
ali – Ada ali-files
bin – binary files
lib - lib-files
license.txt
makefile
obj – object files after Gnatmake compilation
pre – ADA files to be preprocessed by y2018text preprocessor
Read.me
sample – the sample
src – ADA source code files
tmp – temporary files